ESTUDOS DE CASO • AULA 13 • CONSTRUTORES E INSTÂNCIAS

Objetos que nascem certos

Dois estudos de caso com validação no construtor, throw e try/catch

Duração total	Formato	Linguagem
1h30 (40 min + 50 min)	Individual ou em duplas	TypeScript ou JavaScript, conforme a turma

O que você vai praticar

- Criar classes com `constructor`, `this` e **valores padrão** nos parâmetros.
- Validar os dados no construtor e recusar dados inválidos com `throw new Error(...)`.
- Tentar criar objetos dentro de `try` e tratar a recusa no `catch`, sem o programa parar.
- Guardar instâncias em listas e percorrê-las com `for...of`.
- Perceber que cada instância tem seus próprios dados, inclusive suas próprias listas.

Lembrete rápido: throw e try/catch

O construtor usa `throw new Error("mensagem")` para recusar um dado inválido. Quem cria o objeto usa `try`/`catch` para tratar essa recusa e continuar o programa:

Modelo • TypeScript e JavaScript (igual nas duas)

```
try {
  const objeto = new MinhaClasse(/* dados */);
  // só chega aqui se o construtor não lançou erro
} catch (erro) {
  if (erro instanceof Error) {
    console.log(erro.message);
  }
}
```

Quando o construtor lança um erro, a criação é interrompida: nenhuma instância é entregue, e a execução pula direto para o `catch`.

Roteiro e tempo sugerido

Etapa	Tempo	O que você entrega
Caso 1 • Leitura e planejamento	5 min	Regras do ingresso entendidas
Caso 1 • Parte 1: a classe Ingresso	15 min	Classe com validações e três testes
Caso 1 • Parte 2: os pedidos do dia	15 min	Laço com `try`/`catch` para cada pedido
Caso 1 • Parte 3: fechamento do caixa	5 min	Quantidade vendida e total arrecadado
Caso 2 • Leitura e planejamento	5 min	Regras do médico e da agenda entendidas
Caso 2 • Parte 1: a classe Medico	15 min	Construtor com padrão, validações e lista própria
Caso 2 • Parte 2: agendamentos	15 min	Método `agendar` com validações e tentativas tratadas
Caso 2 • Parte 3: conferência	10 min	Resumo e lista de pacientes de cada médico

Perguntas para conversar	5 min	Respostas às cinco perguntas
Total	**90 min**	

ESTUDO DE CASO 1 • 40 MINUTOS

Cine Bairro: venda de ingressos

O **Cine Bairro** é um cinema pequeno, com uma sala de 120 lugares. Os pedidos de ingresso chegam pelo balcão e pelo site, e alguns vêm com erros: filme em branco, assento que não existe, tipo de ingresso desconhecido. Hoje, quando isso acontece, o sistema antigo trava e ninguém consegue vender mais nada.

Você vai escrever a classe que representa um ingresso. Ela só deve deixar nascer ingressos válidos. O programa deve recusar os pedidos com problema, explicar o motivo e **continuar** atendendo os próximos.

Regras do ingresso

Dado	Regra	Mensagem se a regra falhar
`filme`	Texto obrigatório. Remova os espaços das pontas com `trim()`; se sobrar texto vazio, recuse.	Filme obrigatório.
`assento`	Número inteiro de 1 a 120.	Assento inválido.
`tipo`	`"inteira"` ou `"meia"`. Se não for informado, vale `"inteira"`.	Tipo de ingresso inválido.

Tipo	Preço
inteira	R$ 32,00
meia	R$ 16,00

Valide os dados **na ordem da tabela** (filme, assento, tipo). Assim, um pedido com mais de um problema mostra sempre a mesma mensagem.

CASO 1 • PARTE 1 • 15 MINUTOS

A classe Ingresso

1. Crie a classe `Ingresso` com os atributos `filme`, `assento` e `tipo`.
2. Escreva o `constructor(filme, assento, tipo)`, com `"inteira"` como valor padrão de `tipo`. Valide os três dados com as regras da tabela, usando `throw new Error(...)`, e só depois guarde os valores com `this`. Guarde o filme já sem os espaços das pontas.
3. Crie o método `calcularPreco()`, que retorna 32 ou 16 conforme o tipo.
4. Crie o método `descrever()`, que retorna um texto no formato da saída abaixo, com o preço em duas casas decimais (`toFixed(2)`).
5. Teste: crie um ingresso inteira para o filme “O Último Farol”, assento 10, e um meia para o assento 11. Mostre a descrição dos dois. Depois, dentro de `try`/`catch`, tente criar um ingresso para o assento 130 e mostre `Erro:` seguido da mensagem.

SAÍDA ESPERADA NO TERMINAL

```
O Último Farol | assento 10 | inteira | R$ 32.00
O Último Farol | assento 11 | meia | R$ 16.00
Erro: Assento inválido.
```

CASO 1 • PARTE 2 • 15 MINUTOS

Os pedidos do dia

Copie a lista de pedidos abaixo para o seu arquivo, depois da classe. Cada pedido traz os dados de um ingresso; quando não há `tipo`, o padrão deve ser usado.

Pedidos do dia • TypeScript e JavaScript (igual nas duas)

```
const pedidos = [
  { filme: "O Último Farol", assento: 20 },
  { filme: "O Último Farol", assento: 21, tipo: "meia" },
  { filme: "   ", assento: 22 },
  { filme: "Viagem a Marte", assento: 0 },
  { filme: "Viagem a Marte", assento: 45, tipo: "estudante" },
  { filme: "Viagem a Marte", assento: 46, tipo: "meia" },
  { filme: "Viagem a Marte", assento: 7.5 }
];
```

1. Crie uma lista vazia chamada `vendidos` para guardar os ingressos aprovados.
2. Percorra `pedidos` com `for...of`. Para cada pedido, **dentro de um `try`**, crie o ingresso com `new Ingresso(pedido.filme, pedido.assento, pedido.tipo)`, guarde-o em `vendidos` e mostre a mensagem de aprovação.
3. No `catch`, mostre que o pedido foi recusado e o motivo.
4. Numere os pedidos a partir de 1, como na saída abaixo.

SAÍDA ESPERADA NO TERMINAL

```
Pedido 1 aprovado: O Último Farol | assento 20 | inteira | R$ 32.00
Pedido 2 aprovado: O Último Farol | assento 21 | meia | R$ 16.00
Pedido 3 recusado: Filme obrigatório.
Pedido 4 recusado: Assento inválido.
Pedido 5 recusado: Tipo de ingresso inválido.
Pedido 6 aprovado: Viagem a Marte | assento 46 | meia | R$ 16.00
Pedido 7 recusado: Assento inválido.
```

PENSE

O pedido 3 deu erro, mas o pedido 4 foi processado. Por quê? O que aconteceria se a criação do ingresso estivesse fora do `try`?

CASO 1 • PARTE 3 • 5 MINUTOS

Fechamento do caixa

Percorra a lista `vendidos`, some o preço de cada ingresso e mostre a quantidade vendida e o total arrecadado.

SAÍDA ESPERADA NO TERMINAL

```
Ingressos vendidos: 3
Total arrecadado: R$ 64.00
```

CONFIRA

Foram aprovados os pedidos 1 (inteira), 2 (meia) e 6 (meia): 32 + 16 + 16 = 64. Os pedidos recusados nunca entraram na lista, porque nenhum ingresso chegou a ser criado para eles.

ESTUDO DE CASO 2 • 50 MINUTOS

Clínica Bem-Estar: agenda de consultas

A **Clínica Bem-Estar** quer organizar a agenda do dia de cada médico. Cada médico atende um número máximo de pacientes por dia. Quando a agenda enche, a recepção precisa saber na hora, sem que o sistema pare de funcionar para os outros médicos.

Você vai escrever a classe `Medico`. Ela valida os dados no cadastro e também ao agendar um paciente.

Regras

Momento	Regra	Mensagem se a regra falhar
Cadastro (construtor)	`nome` obrigatório, sem espaços nas pontas.	Nome do médico obrigatório.
Cadastro (construtor)	`limiteDiario`: inteiro de 1 a 10. Se não for informado, vale 3.	Limite diário inválido.
Cadastro (construtor)	Cada médico começa com sua própria lista de pacientes, vazia.	—
Agendamento	Nome do paciente obrigatório, sem espaços nas pontas.	Paciente obrigatório.
Agendamento	Não pode passar do limite diário do médico.	Agenda lotada: nome do médico.

CASO 2 • PARTE 1 • 15 MINUTOS

A classe Medico

1. Crie a classe `Medico` com os atributos `nome`, `especialidade`, `limiteDiario` e `pacientes` (uma lista de nomes).
2. Escreva o `constructor(nome, especialidade, limiteDiario)`, com 3 como valor padrão do limite. Valide nome e limite com `throw new Error(...)`, guarde os valores com `this` e crie a lista de pacientes **vazia, dentro do construtor**.
3. Crie o método `resumo()`, que retorna o texto no formato da saída abaixo: nome, especialidade e ocupação (pacientes agendados / limite).
4. Teste: cadastre a Dra. Lúcia (Cardiologia, limite 2) e o Dr. Paulo (Pediatria, limite padrão) e mostre o resumo dos dois. Dentro de `try`/`catch`, tente cadastrar a Dra. Rita (Dermatologia, limite 0) e mostre `Cadastro recusado:` seguido da mensagem.

SAÍDA ESPERADA NO TERMINAL

```
Lúcia (Cardiologia): 0/2 consultas
Paulo (Pediatria): 0/3 consultas
Cadastro recusado: Limite diário inválido.
```

CASO 2 • PARTE 2 • 15 MINUTOS

Agendamentos do dia

1. Crie o método `agendar(paciente)`. Ele valida o nome do paciente e o limite com `throw new Error(...)` e, se estiver tudo certo, acrescenta o paciente (sem espaços nas pontas) à lista do médico com `push`.
2. Faça as cinco tentativas da tabela abaixo, nesta ordem. Cada tentativa deve ficar em seu próprio `try`/`catch`, para que uma recusa não impeça as seguintes.

Tentativa	Médico	Paciente
1	Lúcia	Ana
2	Lúcia	Bruno
3	Lúcia	Carla
4	Paulo	" " (só espaços)
5	Paulo	Davi

SAÍDA ESPERADA NO TERMINAL

```
Agendado: Ana com Lúcia
Agendado: Bruno com Lúcia
Não agendado: Agenda lotada: Lúcia.
Não agendado: Paciente obrigatório.
Agendado: Davi com Paulo
```

DICA

As cinco tentativas repetem o mesmo `try`/`catch`. Você pode escrever uma função, por exemplo `tentarAgendar(medico, paciente)`, que faz a tentativa e mostra a mensagem. Não é obrigatório: repetir o bloco também está correto.

CASO 2 • PARTE 3 • 10 MINUTOS

Conferência

Coloque os dois médicos em uma lista chamada `equipe`. Percorra a lista e, para cada médico, mostre o resumo e, logo abaixo, os nomes dos seus pacientes, um por linha, com um recuo e um traço.

SAÍDA ESPERADA NO TERMINAL

```
Lúcia (Cardiologia): 2/2 consultas
  - Ana
  - Bruno
Paulo (Pediatria): 1/3 consultas
  - Davi
```

CONFIRA

Carla não aparece na lista de Lúcia, e o paciente em branco não aparece na de Paulo: as tentativas recusadas não mudaram nada. A lista de Paulo também não tem Ana nem Bruno, porque cada médico tem a sua própria lista.

Perguntas para conversar • 5 minutos

1. No caso 1, o que aconteceria com os pedidos 4 a 7 se a criação do ingresso estivesse fora do `try`?
2. No caso 2, quantos objetos `Medico` existem ao final? A tentativa de cadastrar a Dra. Rita conta?
3. Por que a lista de pacientes é criada com `this.pacientes = []` dentro do construtor? O que mudaria se todos os médicos recebessem a mesma lista?
4. Se escrevêssemos `const plantao = lucia;` e depois tentássemos agendar Eva em `plantao`, o que aconteceria? E se fosse `new Medico("Lúcia", "Cardiologia", 2)`?
5. Qual a diferença entre o erro lançado no construtor e o erro lançado em `agendar`? Em qual dos dois casos o objeto já existe?

Checklist de entrega

- Arquivo do caso 1 (`cinema.ts` ou `cinema.js`), com a classe `Ingresso`, os pedidos e o fechamento.
- Arquivo do caso 2 (`clinica.ts` ou `clinica.js`), com a classe `Medico`, os agendamentos e a conferência.
- Saídas do terminal iguais às saídas esperadas.
- Respostas às perguntas para conversar.